Develop a program to implement the Naive Bayesian classifier considering Iris dataset for training. Compute the accuracy of the classifier, considering the test data.

Aim:

To develop a program to implement the Naive Bayes classifier using the Iris dataset and compute the accuracy of the classifier using test data.

Theory:

The **Naive Bayes Classifier** is a probabilistic supervised learning algorithm based on **Bayes' Theorem**, which is used to determine the probability of a data point belonging to a particular class. It is widely used for classification tasks due to its simplicity and efficiency.

Bayes' Theorem is defined as:

$P(C|X)=P(X|C)\ P(C)P(X)/P(C|X)$

here:

- P(C|X) is the posterior probability of class C given feature vector X
- P(X|C) is the likelihood of features given the class
- P(C) is the prior probability of the class
- P(X) is the evidence

The classifier is called "naive" because it assumes that all input features are **conditionally independent** given the class label. This means the presence or absence of one feature does not affect another feature. Although this assumption is not always true, it simplifies computation and works well in many practical cases.

In this experiment, the **Iris dataset** is used, which consists of 150 samples belonging to three different classes of iris flowers: *Setosa, Versicolor, and Virginica*. Each sample has four features:

- Sepal length
- Sepal width
- Petal length
- Petal width

The dataset is divided into **training and testing sets**. The training set is used to build the model, while the testing set is used to evaluate its performance.

A **Gaussian Naive Bayes classifier** is applied, which assumes that the continuous features follow a **normal (Gaussian) distribution**. For each feature, the mean and variance are calculated for every class during training. Using these values, the probability of a data point belonging to each class is computed.

During prediction, the classifier calculates the **posterior probability** for each class and assigns the class with the highest probability to the given input.

The performance of the classifier is evaluated using **accuracy**, which is defined as:

Accuracy= Total Number of Predictions/Number of Correct Predictions

Naive Bayes is highly efficient, requires less training data, and performs well even with high-dimensional datasets, making it suitable for real-time classification problems.

[7]:
```python
# Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X = iris.data        # Features
y = iris.target      # Target labels

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Create Naive Bayes model
model = GaussianNB()

# Train the model
model.fit(X_train, y_train)

# Predict on test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

# Print result
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9333333333333333
```

Result:

The Naive Bayes classifier was successfully implemented using the Iris dataset, and the accuracy of the model was calculated using test data.